MAQ Software

.NET, ASP .NET Core & MVC Bootcamp

Pooja Ghagare

October 9, 2026











Microsoft Power BI
Partner of the Year 2021

snowflake
Technology Partner

databricks
Technology Partner



Inc. 5000

We exist to build great software

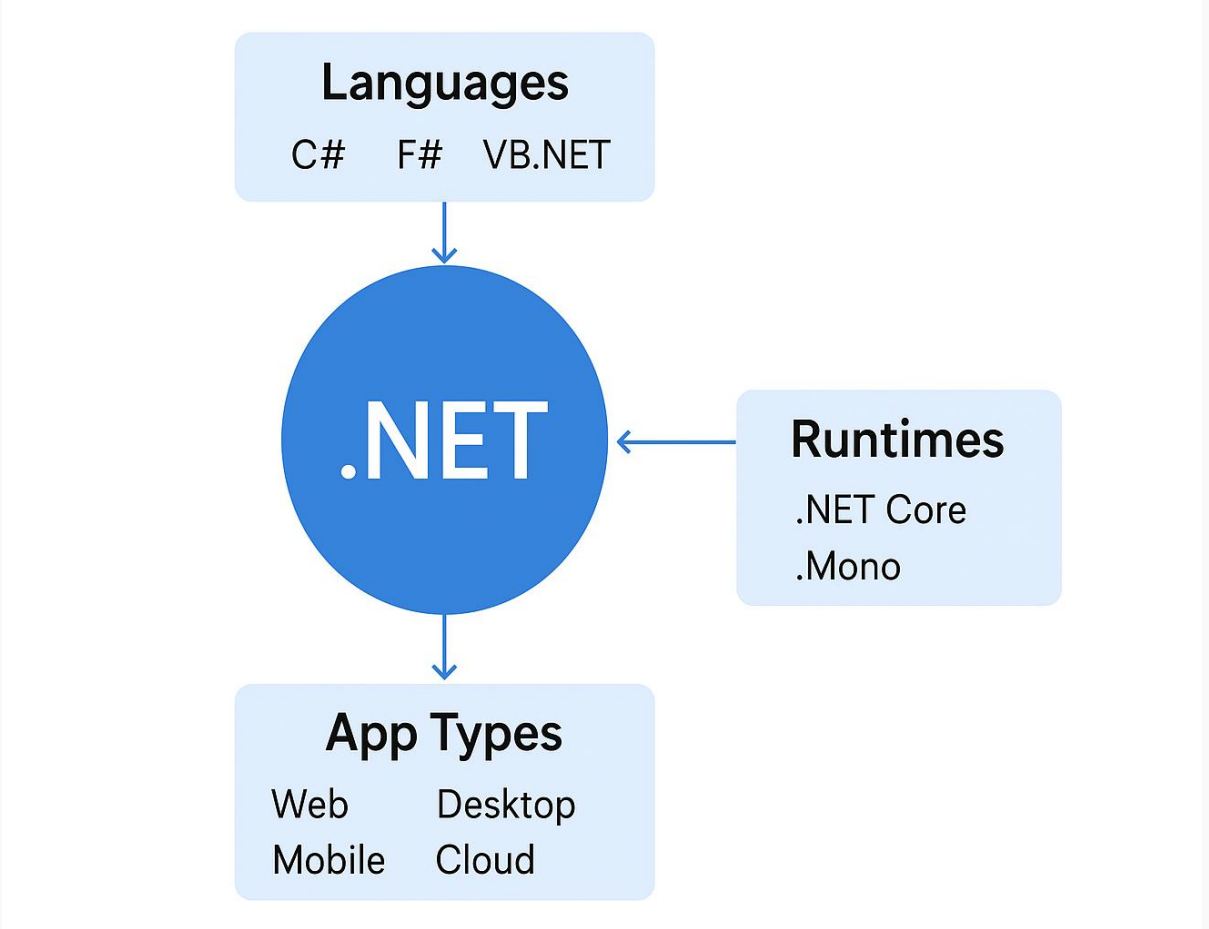
Agenda

- .NET Fundamentals & Project Structure
- CLI, NuGet, and Starting an ASP.NET Project
- MVC, Routing & CRUD
- Dependency Injection & Authentication Basics

.NET Fundamentals & Project Structure

What is .NET?

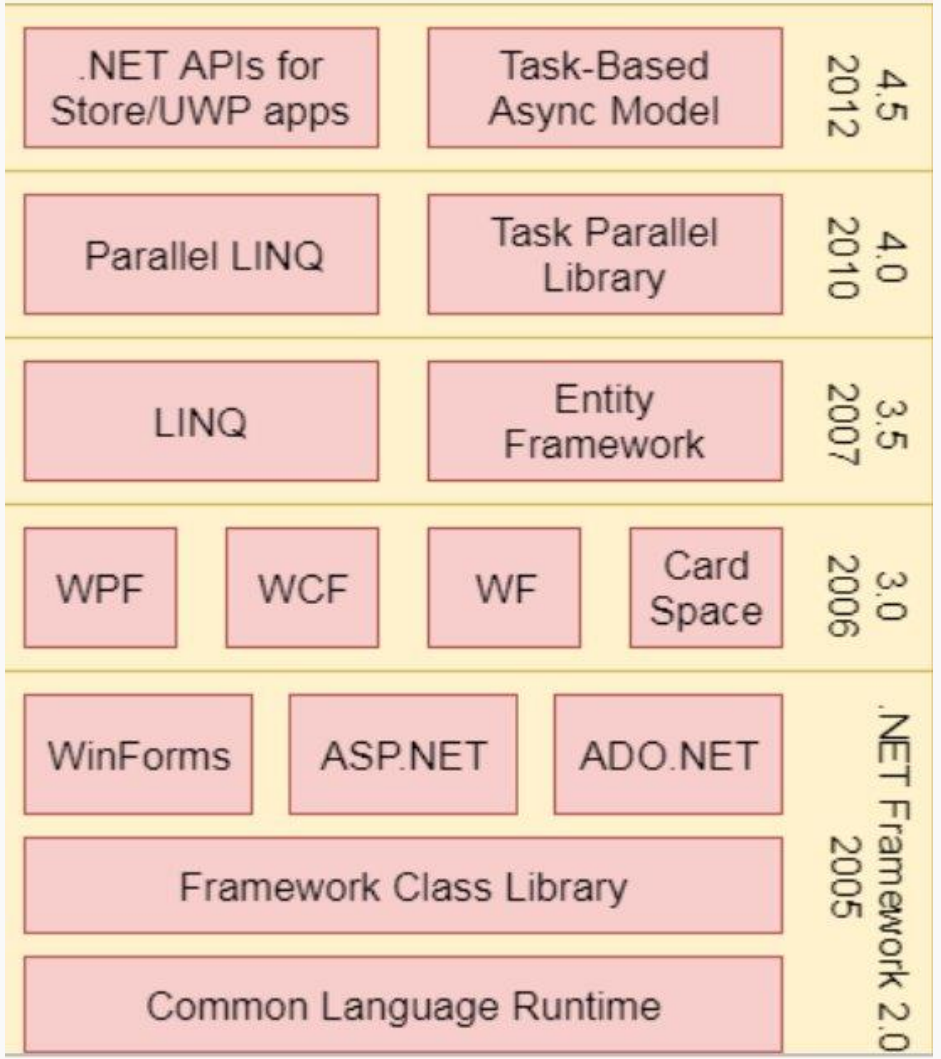
- **Definition:** A cross-platform, open-source framework for building applications across web, mobile, desktop, and cloud.
- **Ecosystem Structure:** Comprises programming languages (C#, F#, VB.NET), runtimes (e.g., .NET Core, Mono), and multiple app types.
- **Key Notes:** From .NET 5 onward, it is a unified platform enabling consistent development experiences across devices and operating systems.



.NET Core vs .NET 5/6+

- **.NET Core (1–3.1)**
  - **Modular:** Lightweight and allows adding only the required libraries.
  - **Cross-Platform:** Runs on Windows, Linux, and macOS.
  - **Open Source:** Community-driven with support from Microsoft.
  - **Use Case:** Ideal for microservices and modern web APIs.

- **.NET 5/6+**
  - **Unified Platform:** Combines .NET Framework, .NET Core, and Xamarin under one runtime.
  - **LTS Releases:** Long-Term Support ensures stability for enterprise apps.
  - **Performance Gains:** Faster startup, reduced memory usage, and improved scalability.



.csproj & Project Structure

- **.csproj (C# Project File)**
  - An XML file that stores all configuration details for a C# project.
  - Contains metadata like **project type**, **target framework**, **package references**, and **build settings**.
  - Automatically updated by Visual Studio or CLI when you add/remove dependencies.

- **Key Elements**
  - <TargetFramework> — Specifies the version of .NET your app is targeting (e.g., net6.0).
  - <PackageReference> — Declares external NuGet packages your project depends on (e.g., Newtonsoft.Json).

- **Project Folder Structure** *(ASP.NET Core MVC Example)*
  - **Controllers/** — Contains classes that handle incoming requests and return responses.
  - **Models/** — Holds data classes and business logic.
  - **Views/** — Contains Razor view files (.cshtml) for UI rendering.
  - **wwwroot/** — Public static files like CSS, JavaScript, and images.
  - **appsettings.json** — Stores configuration values (DB connections, API keys, etc.).

CLI, NuGet & Project Setup

.NET CLI Essentials

- **dotnet new — Create a Project**
  - Creates a new .NET project, solution, or file from a template.
  - Example: dotnet new mvc -n MyApp → Creates an ASP.NET Core MVC project named *MyApp*.

- **dotnet build — Compile the Project**
  - Compiles the application and generates binaries (DLLs).
  - Example: dotnet build → Builds the current project.

- **dotnet run — Execute the Project**
  - Builds (if not already built) and runs the application.
  - Example: dotnet run → Runs the app from the current folder.

NuGet - Package manager for .NET

Purpose

- Central package manager for .NET libraries — makes it easy to discover, install, and manage third-party and internal libraries.
- Handles dependency resolution, versioning, and distribution so you don't ship library source with your app.

Example

- dotnet add package Newtonsoft.Json
  - Adds the Newtonsoft.Json NuGet package to the current project and updates the project file (<PackageReference>).
  - After adding, dotnet restore (or dotnet build) downloads the package into the local cache.

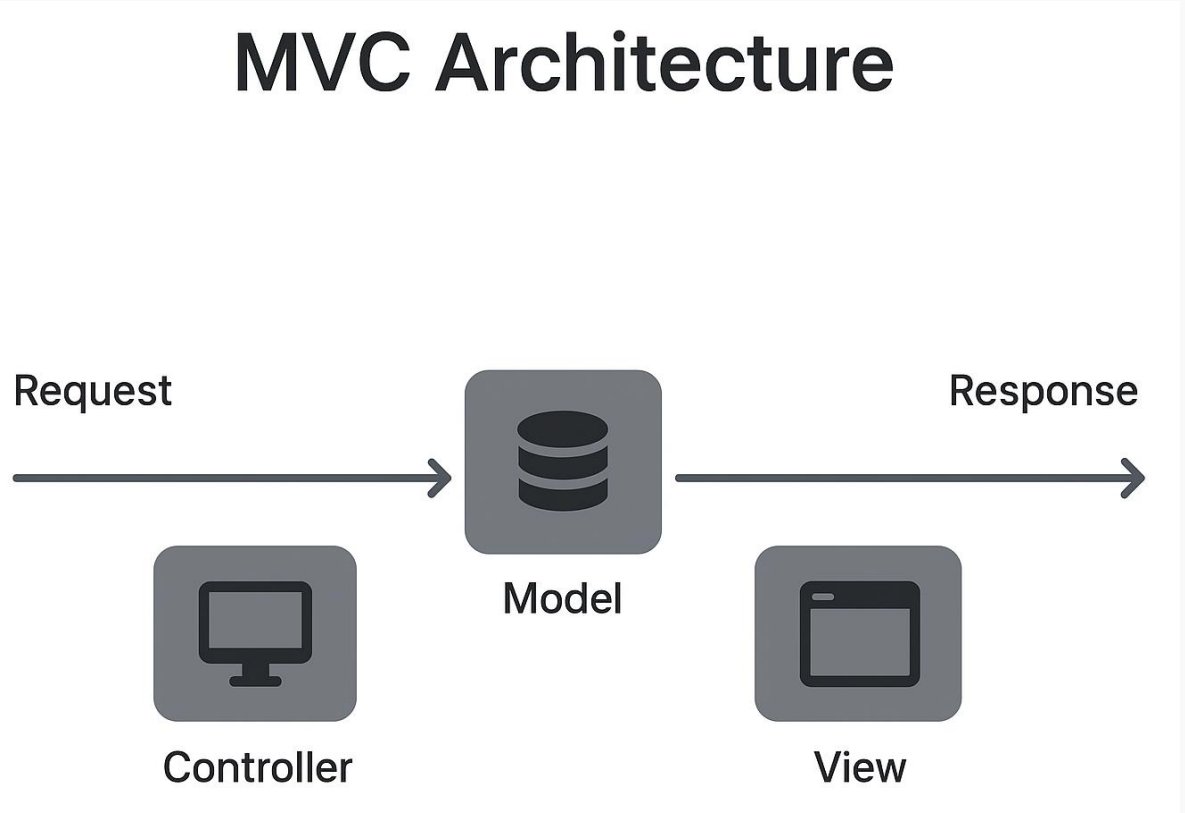
Creating ASP.NET Core Project

- **CLI Command**
- dotnet new mvc -n MyApp
  - Creates a new ASP.NET Core MVC project named MyApp.
  - Generates the default folder hierarchy and boilerplate files.
- **Folder Breakdown**
  - **Controllers** — Handle user input, business logic, and select the view to render.
  - **Views** — HTML templates (Razor .cshtml files) used to present data.
  - **Models** — Represent application data and business rules.
  - **wwwroot** — Static files (CSS, JS, images) served directly to the client.

MVC, Routing & CRUD

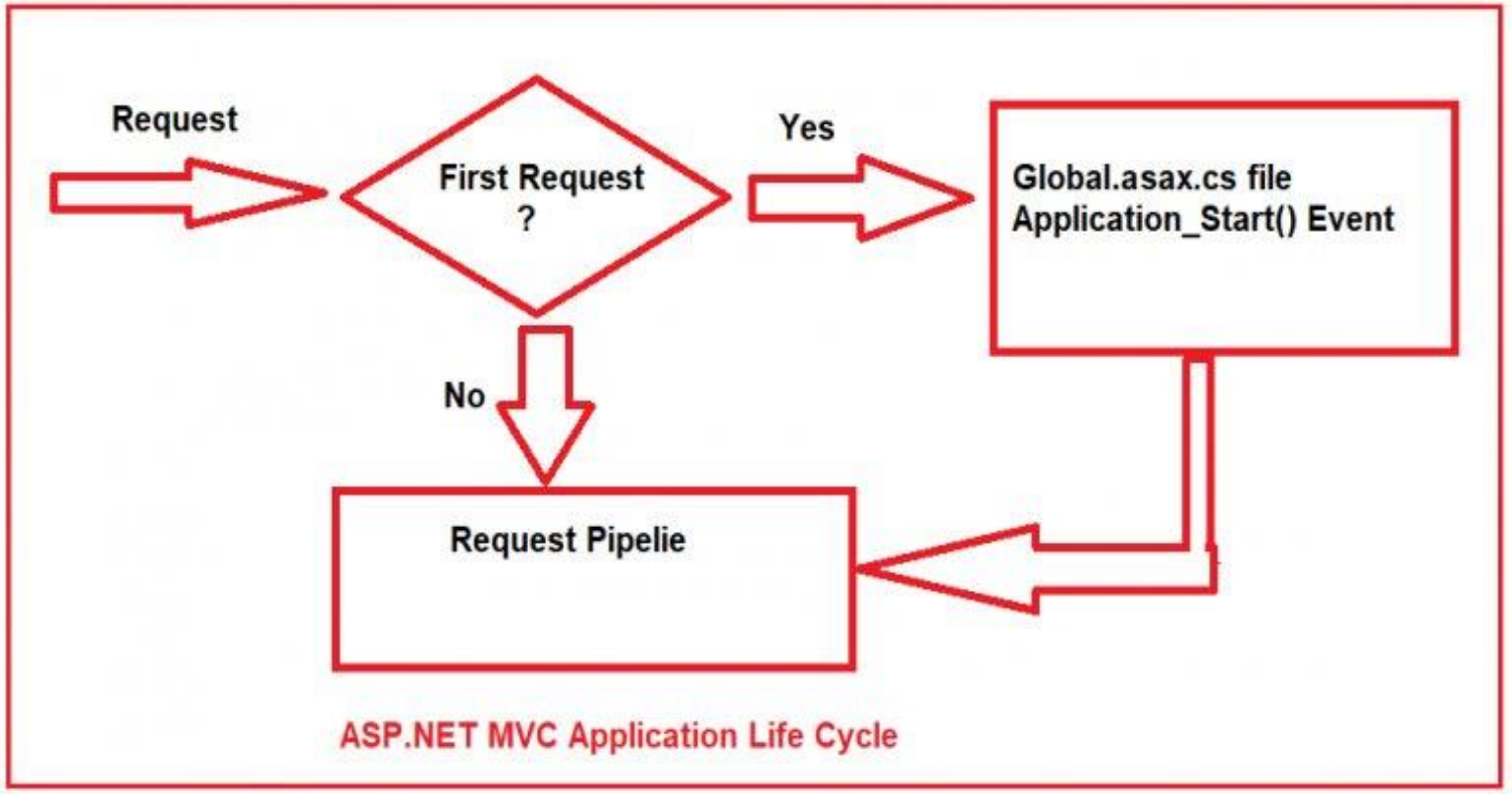
MVC Architecture

- **Model**
  - Holds application data and business rules.
  - Interacts with the database or data source.
  - Example: Product class, Entity Framework models.
- **View**
  - Represents the presentation layer (UI).
  - Displays data from the model to the user.
  - Example: .cshtml Razor views in ASP.NET Core.
- **Controller**
  - Acts as the middleman between Model and View.
  - Handles incoming HTTP requests, processes data via the model, and selects the appropriate view to render.
  - Example: HomeController.cs.



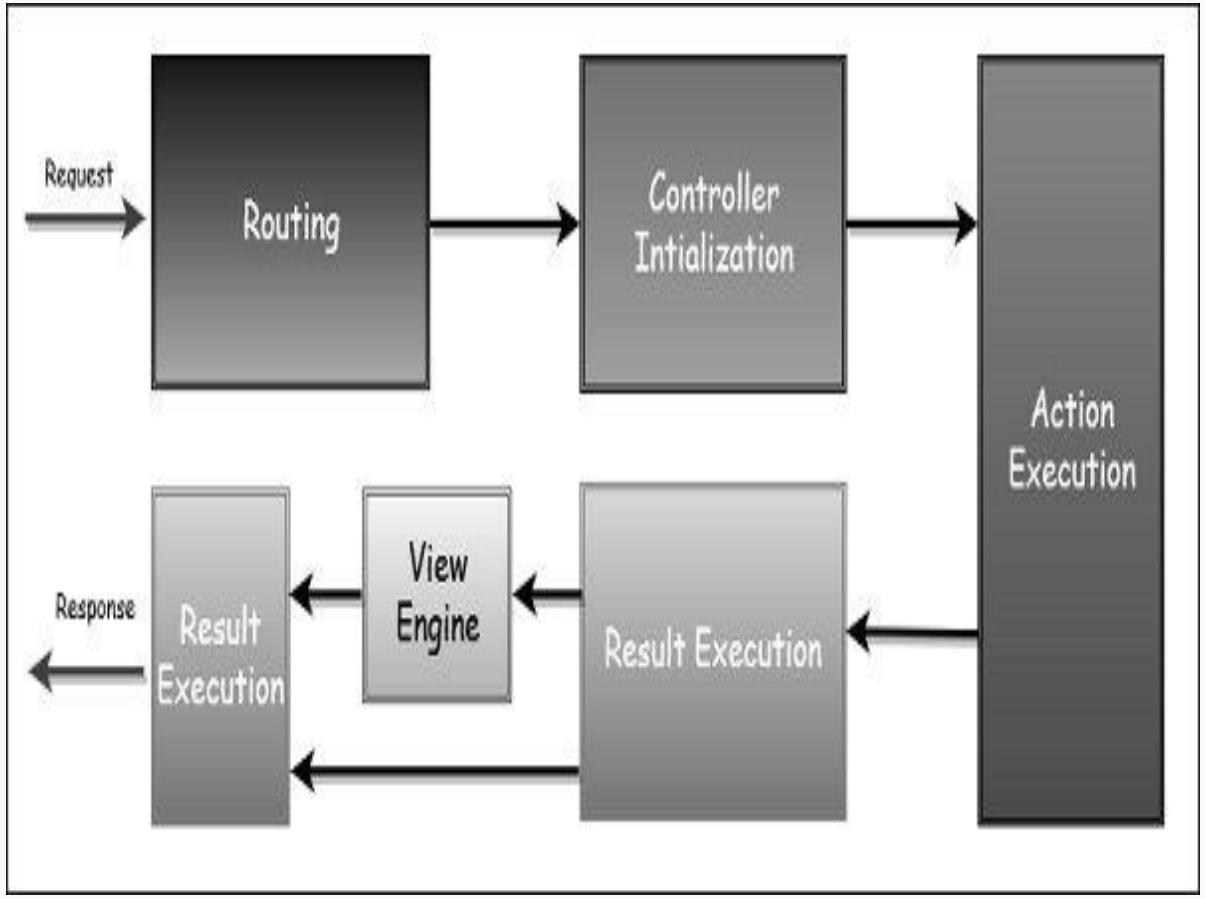
MVC – Application Life Cycle

The application life cycle refers to the time at which the application process begins running IIS until the time it stops. This is marked by the application start and end events in the startup file of your application.



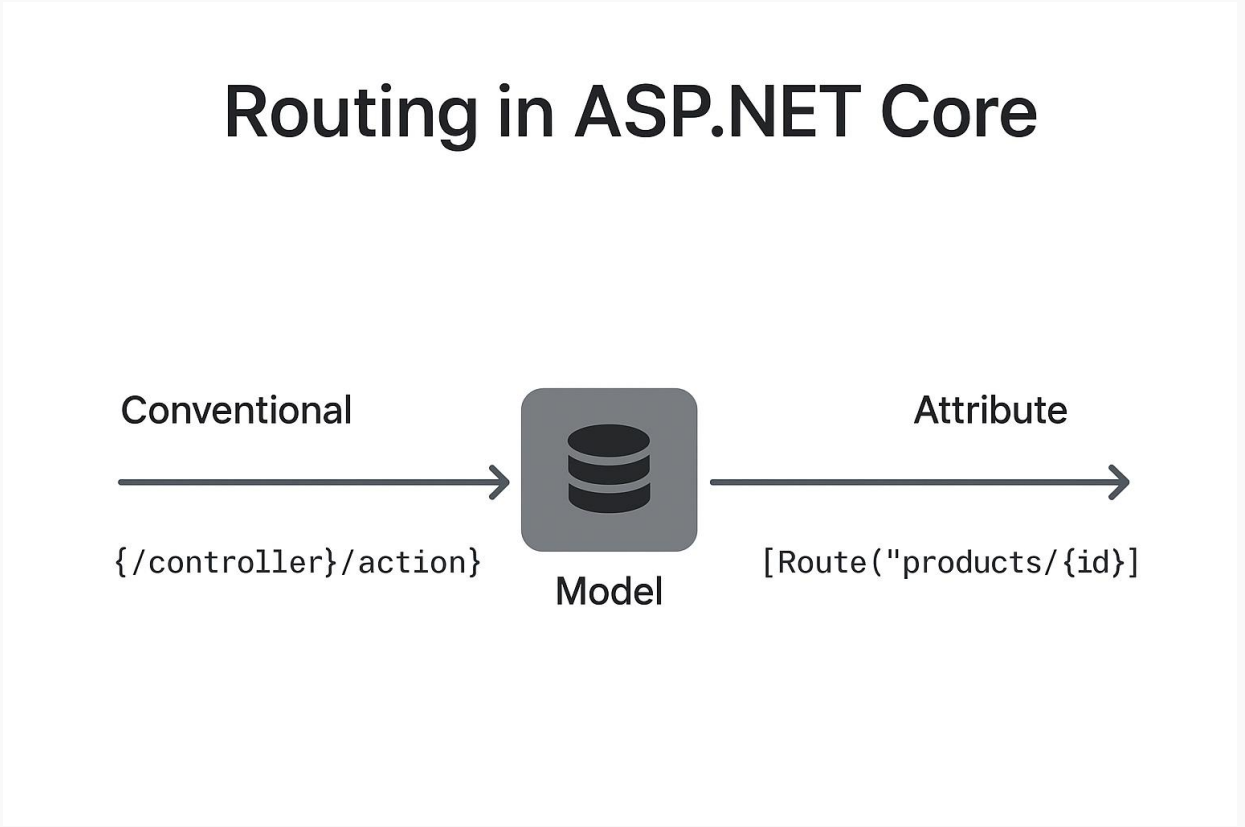
MVC – Request Life Cycle

- It is the sequence of events that happen every time an HTTP request is handled by our application.
- The entry point for every MVC application begins with routing. After the ASP.NET platform has received a request, it figures out how it should be handled through the URL Routing Module.
- Modules are .NET components that can hook into the application life cycle and add functionality. The routing module is responsible for matching the incoming URL to routes that we define in our application.
- All routes have an associated route handler with them, and this is the entry point to the MVC framework.



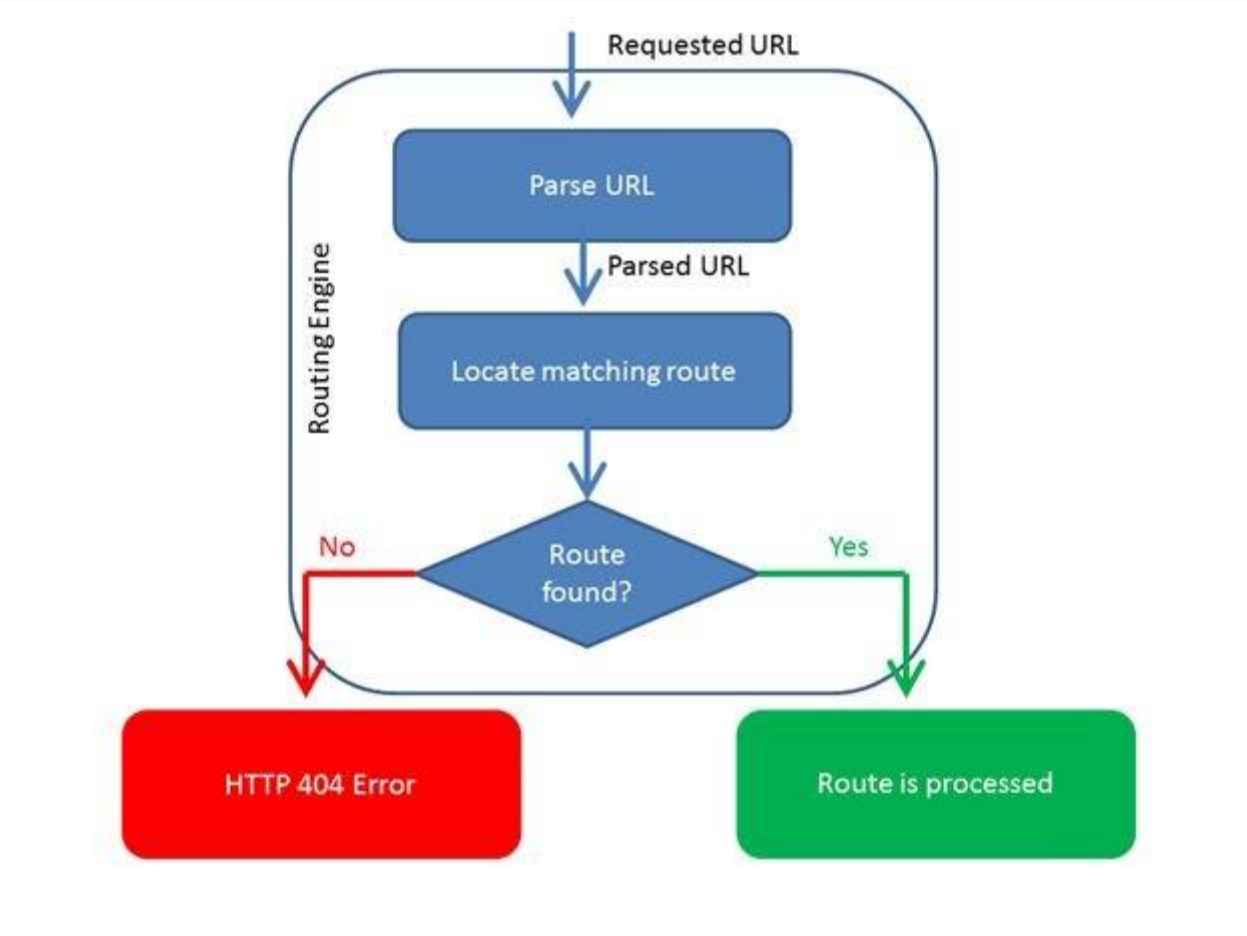
Routing

- **Conventional Routing** (/{controller}/{action}/{id?})
  - Uses a predefined URL pattern to map incoming requests to a controller and action.
  - {id?} is optional, meaning the route will work with or without it.
  - Example: /products/details/5 → ProductsController.Details(5).
- **Attribute Routing** ([Route("products/{id}")])
  - Defines routes directly on controller actions using attributes.
  - Gives more control over URL structure.
  - Example: [Route("products/{id}")] → matches /products/5 regardless of default route patterns.



Routing

- **Properties of Routes:**
  - ASP.NET MVC routes are responsible for determining which controller method to execute for a given URL. A URL consists of the following properties:
    - Route Name
    - URL Pattern
    - Defaults
    - Constraints
- **Request URL Mapping Flow** (Diagram)
  - **Step 1:** User sends a request URL.
  - **Step 2:** The routing system matches it against defined routes.
  - **Step 3:** If matched, it identifies the target controller and action.
  - **Step 4:** Controller processes the request, interacts with the model if needed.
  - **Step 5:** Returns the appropriate view or data as a response.



Entity Framework and CRUD

What is EF Core?

- Let's you work with a **database using .NET objects** instead of writing raw SQL queries.
- It maps your **C# classes (entities)** to **database tables** and **object properties** to **columns**.
- Supports multiple database providers like **SQL Server, SQLite, PostgreSQL, MySQL**, etc.

Highlight:

EF Core acts like a **translator** between your C# code and the database, allowing you to focus on business logic instead of low-level SQL commands.

CRUD

- **C**reate → Add new data to the database.
- **R**ead → Retrieve existing data.
- **U**pdate → Modify existing data.
- **D**elete → Remove data from the database.
- This is the fundamental cycle for database operations in most applications.

Entity Framework - CRUD Example

- Create (Insert Data)

```
var user = new User { Name = "John", Email = "john@example.com" };
dbContext.Users.Add(user);       // Stage insert operation
dbContext.SaveChanges();         // Commit to DB
```

- Read (Fetch Data)

```
var users = dbContext.Users.ToList();                 // Get all users
var singleUser = dbContext.Users.FirstOrDefault(u => u.Id == 1);  // Get one user by condition
```

- Update (Modify Data)

```
var userToUpdate = dbContext.Users.First(u => u.Id == 1);
userToUpdate.Name = "John Doe";
dbContext.Users.Update(userToUpdate);   // Stage update
dbContext.SaveChanges();                // Commit changes
```

- Delete (Remove Data)

```
var userToDelete = dbContext.Users.First(u => u.Id == 1);
dbContext.Users.Remove(userToDelete);   // Stage delete
dbContext.SaveChanges();                // Commit delete
```

Dependency Injection & Authentication

Dependency Injection (DI)

- **What is Loose Coupling via Service Injection?**
  - Think of your application as a factory.
  - The **Controller** is the manager — it tells workers what to do.
  - The **Service Interface** is the job description.
  - The **Service Implementation** is the actual worker doing the job.
  - If the manager **only knows the job description** (interface), you can swap workers (implementations) without retraining the manager.

- **Why do we use it?**
  - **Flexibility:** Change how a task is done without changing the controller code.
  - **Testability:** Swap real services with fake/mock services for testing.
  - **Clean Code:** Each part of your app focuses on one responsibility.

- Example:If you order pizza:
  - You call a **Pizza Service** (interface) — "I want pizza."
  - Today, a local pizza shop (implementation) makes it.
  - Tomorrow, a different shop could deliver without you changing how you place the order

Dependency Injection (DI) Example

```
// 1. Register service in Program.cs
services.AddScoped<IEmailService, EmailService>();

// 2. Define an interface
public interface IEmailService
{
    void Send(string to, string message);
}

// 3. Implement the service
public class EmailService : IEmailService
{
    public void Send(string to, string message)
    {
        Console.WriteLine($"Email sent to {to}: {message}");
    }
}
}
```

```
// 4. Use it in a controller
public class NotificationController : Controller
{
    private readonly IEmailService _emailService;

    public NotificationController(IEmailService emailService)
    {
        _emailService = emailService;
    }

    public IActionResult SendMail()
    {
        _emailService.Send("test@example.com", "Hello");
        return Ok();
    }
```

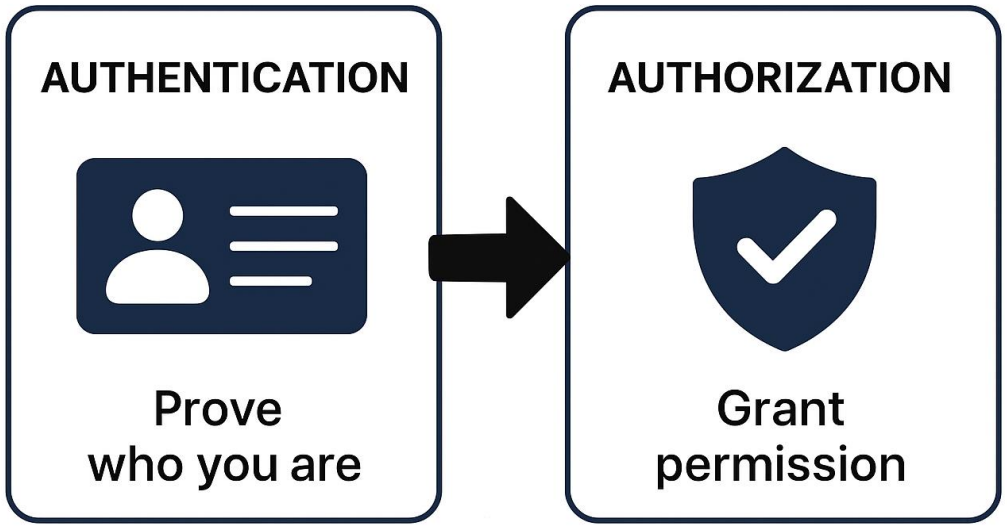
Authentication vs Authorization

1. Authentication

- **What it is:** Proving *who you are*.
- **Goal:** Verify the **identity** of a user.
- **Example:**
  - You enter your username & password to log in.
  - The system checks if your credentials are valid.
- Showing your ID card to security at an office gate.

2. Authorization

- **What it is:** Deciding *what you can do*.
- **Goal:** Verify the **permissions** you have.
- **Example:**
  - After logging in, you try to access an admin page.
  - The system checks if you have the right to view it.
- Even if security lets you in the building, you may only have access to certain floors.



Authentication vs Authorization Example

```
// Authentication: Validating identity
services.AddAuthentication("CookieAuth")
    .AddCookie("CookieAuth", config =>
    {
        config.LoginPath = "/Login";
    });

// Authorization: Setting access rules
services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
});
```

Flow:
Authentication → Check if the user is who they claim to be.
Authorization → Check if they're allowed to do the requested action.

MAQ Software

Discussion



Microsoft Premier Supplier
Gold Microsoft Partner
Winner Microsoft Partner 2021 Partner of the Year Power BI Award
amazon web services Partner Network CONSULTING PARTNER
bsi. ISO/IEC 27001 Information Security Management
Inc.5000 The Fastest Growing Private Companies in America